

week_9\week9\src\main\java\week9\StatelessServer.java

```
1 package week9;
2
3 import java.io.*;
4 import java.net.*;
5
6 /**
7  * A simple server for the tic-tac-toe game that listens for client connections and
8  * allows a multi client to play the game against a computer opponent.
9  */
10 public class StatelessServer {
11
12     /**
13      * Main method to start the server and listen for client connections.
14      * @param args
15      * @throws IOException
16      */
17     public static void main(String[] args) throws IOException {
18         try (ServerSocket serverSocket = new ServerSocket(9030)) {
19             while(true){
20                 Socket socket = serverSocket.accept();
21                 System.out.println("Client connected: " + socket.getInetAddress());
22                 handleClient(socket);
23             }
24         } catch(IOException e){
25             System.out.println("Client disconnected: " + e.getMessage());
26         }
27     }
28
29     /**
30      * Handles client connection, check and then reply.
31      * @param socket
32      * @throws IOException
33      */
34     private static void handleClient(Socket socket) throws IOException {
35         // set up streams
36         BufferedReader in = new BufferedReader(new
37             InputStreamReader(socket.getInputStream()));
38         PrintStream out = new PrintStream(socket.getOutputStream(), true);
39
40         //set up computer
41         Computer computer = new Computer(Constants.COMPUTER_MARKER, "COMPUTER");
42
43         //Create board
44         Board board = new Board2D(System.out);
45
46         //Receive client string
47         board.loadString(in.readLine());
48         //String boardString = in.readLine();
49         // Debugging
50         //out.println("Received board: " + boardString + " Length = " +
51         boardString.length());
```

```
50 //Take player move, change from int → position
51 Integer playerMove = Integer.parseInt(in.readLine());
52 Position playerPos = board.getCellPosition(playerMove);
53
54 //Check player move :)
55 if(board.isCellEmpty(playerPos)){
56     //place player move :D
57     board.placeMarker(playerPos, Constants.HUMAN_MARKER);
58 }else{
59     out.println("Cell is occupied!");
60     out.println(board.networkString());
61     return;
62 }
63
64 //Check winners, continue if don't
65 String status = getGameStatus(board);
66
67 if(status.equals("Computer turns")){
68     board.placeMarker(computer.makeMove(board), Constants.COMPUTER_MARKER);
69     status = getGameStatus(board);
70 }
71
72 if(status.equals("wins") || status.equals("Draw!")){
73     out.println(board.networkString());
74     status = getGameStatus(board);
75 }
76
77 out.println(status);
78 out.println(board.networkString());
79 }
80
81 // This is me again, get tired of writing long if else :) so I create a helper
82 private static String getGameStatus(Board board){
83     char winner = board.checkWinner();
84     if (winner == Constants.HUMAN_MARKER) return "You wins!";
85     if (winner == Constants.COMPUTER_MARKER) return "Computer wins!";
86     if (board.isBoardFull()) return "Draw!";
87     return "Computer turns";
88 }
89 }
90
```